Minio对象存储使用简介



中国科学院信息工程研究所
INSTITUTE OF INFORMATION ENGINEERING,CAS



第二研究室 MESA团队
刘俊朋 **liujunpeng@iie.ac.cn**

Minio简介

- Minio

Minio是一个基于Apache 2.0开源对象存储平台，兼容亚马逊S3，由GlusterFS创始人创建。

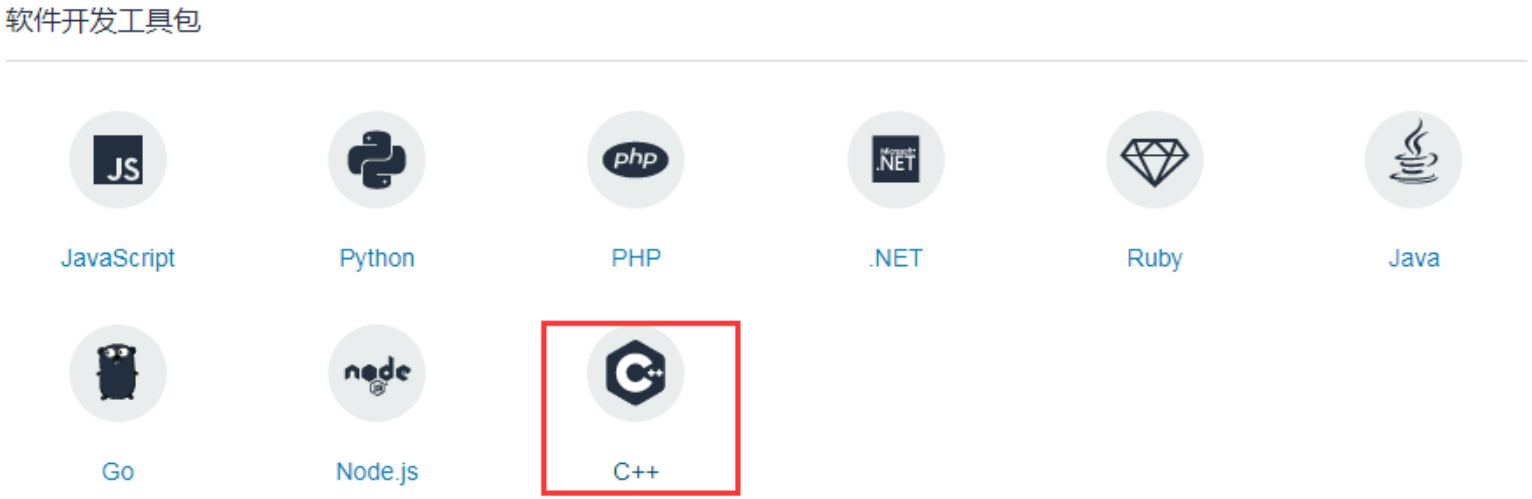
- 用途

Minio非常适用于存储非结构化数据，例如图片、视频、日志文件、备份和容器或虚拟机镜像等。Minio支持从几KB到5TB的对象文件。

AWS S3简介

全称：Amazon Simple Storage Service

是一种面向 Internet 的存储服务。您可以通过 Amazon S3 随时在 Web 上的任何位置存储和检索的任意大小的数据。提供了支持很多语言的软件开发工具包。



本PPT讲述的是利用适用于 C++ 的 AWS 开发工具包，实现在自己部署的分布式Minio环境下来存取文件。

前提条件

- 本PPT讲述的是利用AWS S3提供的C++ API来实现在Minio上存取文件，因此需要实现搭建好Minio环境

- 下面是在4台服务器上部署分布式Minio的启动命令：

  minio server HTTP://192.168.10.61/home/mesasoft/minio/data
  HTTP://192.168.10.62/home/mesasoft/minio/data
  HTTP://192.168.10.63/home/mesasoft/minio/data
  HTTP://192.168.10.64/home/mesasoft/minio/data

安装适用于C++的AWS工具包

1、安装必备软件和库

- yum -y erase cmake
- yum -y install cmake3 gcc-c++ libstdc++-devel libcurl-devel zlib-devel
- cd /usr/bin; ln -s cmake3 cmake
- 可能还会依赖其它库，根据编译时报的错误来判断安装哪个包。

2、准备源码

- cd /tmp
- git clone HTTPs://github.com/AWS/AWS-sdk-cpp.git
- mkdir -p /tmp/build; cd /tmp/build
- cmake -DCMAKE_BUILD_TYPE=Release /tmp/AWS-sdk-cpp

安装适用于C++的AWS工具包

3、编译源码

- make -j `nproc` -C AWS-cpp-sdk-core
- make -j `nproc` -C AWS-cpp-sdk-s3

4、安装头文件和库到一个目录

- mkdir -p /tmp/install
- make install DESTDIR=/tmp/install -C AWS-cpp-sdk-core
- make install DESTDIR=/tmp/install -C AWS-cpp-sdk-s3

注意：安装时用到的目录可以更改，但要注意下面编译代码时指定库的路径

API简要说明

- 代码参考：https://github.com/AWSdocs/AWS-doc-sdk-examples/tree/master/cpp/example_code/s3
- 官方文档：https://AWS.amazon.com/cn/documentation/sdk-for-cpp/

通过AWS提供的适用于C++的工具包，来实现在minio上存取文件，大概分为以下步骤：

1. 初始化AWS接口
2. 建立和minio的连接
3. 具体操作
4. 关闭AWS接口

API简要说明

初始化和关闭AWS接口

```
int main(int argc, char* argv[])
{
        Aws::SDKOptions options;
        Aws::InitAPI(options);

            /*******
             *operate minio
             */

        Aws::ShutdownAPI(options);
        return 0;
}
```

- 图中的代码就是初始化和关闭AWS的API, 后面要讲述的所有对Minio的操作，都是在这之间进行的。

API简要说明

初始化和关闭AWS接口

```
struct SDKOptions
{
        LoggingOptions loggingOptions;
        MemoryManagementOptions memoryManagementOptions;
        HTTPOptions HTTPOptions;
        CryptoOptions cryptoOptions;
};
```

- SDKOptions是一个结构体，里面包含用于记录日志的选项、管理内存的选项、HTTP的选项和加密的选项。
- AWS::InitAPI()是对SDKOptions进行初始化的函数，初始化SDK范围内的状态，在使用这个库执行任何其他操作之前，必须调用这个方法。这个方法主要是对日志级别、内存管理和HTTP这三个选项进行的初始化。

API简要说明

- 创建连接

```
Aws::Client::ClientConfiguration cfg;
cfg.endpointOverride = Aws::String("192.168.10.61:9000"); // minio服务器地址和端口
cfg.scheme = Aws::Http::Scheme::HTTP; //选择http协议

Aws::Auth::AWSCredentials cred("minio", "123456789"); // 封装认证信息，参数是minio服务的MINIO_ACCESS_KEY和MINIO_SECRET_KEY

Aws::S3::S3Client client(cred, cfg, Aws::Client::AWSAuthV4Signer::PayloadSigningPolicy::Never, false); //创建连接
```

上图就是创建连接的几行代码，包括：

a) 封装连接信息
b) 封装认证信息
c) 创建连接

API简要说明

- 创建桶

```
const Aws::String bucket_name = "aaaa"; //定义一个桶名
Aws::S3::Model::CreateBucketRequest request; //创建一个请求创建桶的对象
request.SetBucket(bucket_name); //把桶名封装到请求对象中去
auto outcome = client.CreateBucket(request); //调用CreateBucket函数创建桶
if (outcome.IsSuccess()) //判断是否创建成功
{
        std::cout << "Done!" << std::endl;
}
else
{
        std::cout << "CreateBucket error: "
                << outcome.GetError().GetExceptionName() << std::endl
                << outcome.GetError().GetMessage() << std::endl;
}
```

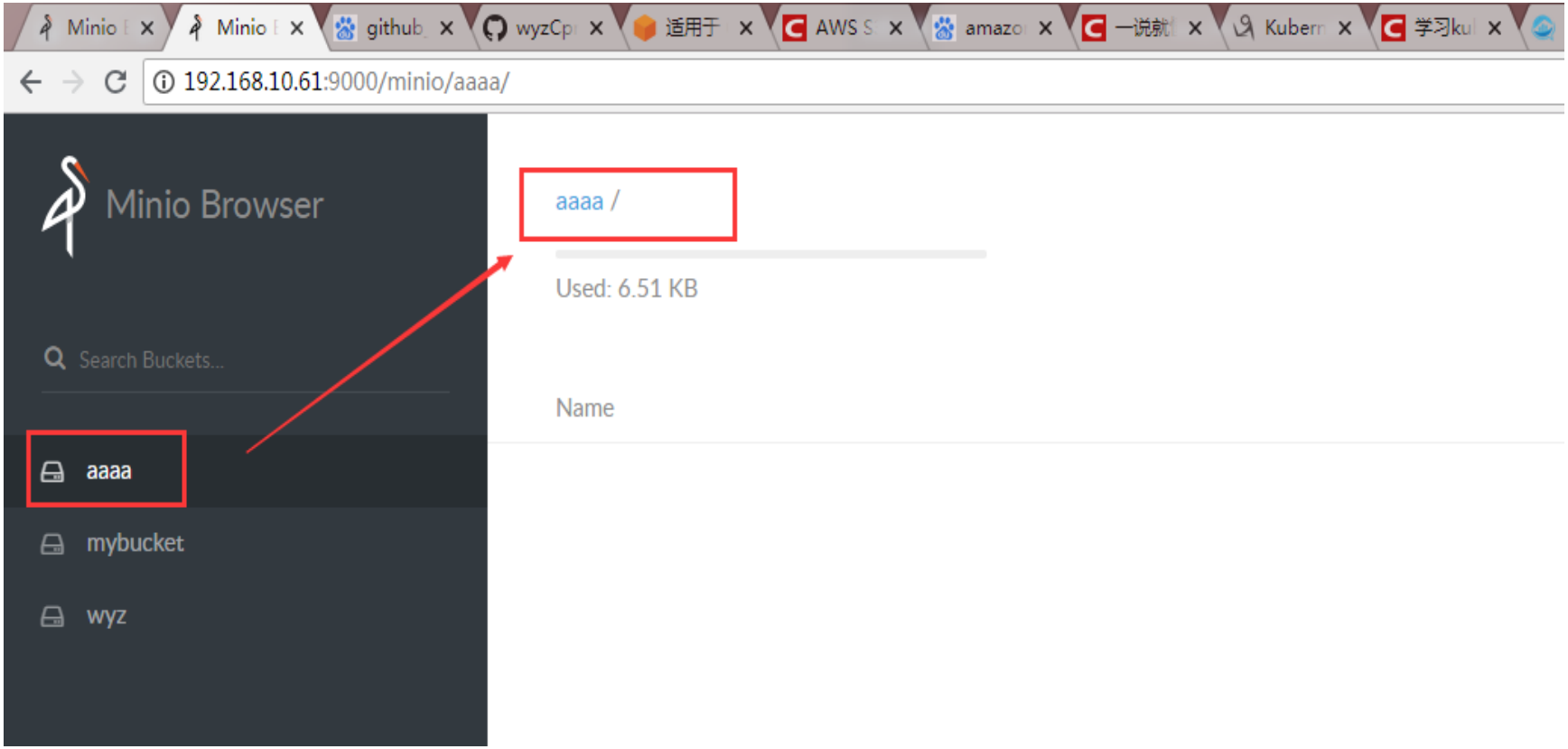
编译并执行，结果如下：

```
[root@master aws]# ./a.out
Done!
[root@master aws]#
```

API简要说明

➢ 创建桶

可以在浏览器中查看桶“aaaa”是否已经创建成功：



API简要说明

➢ 创建桶关于桶名的注意事项

- 存储桶名称的长度介于 3 和 63 个字符之间，并且只能包含小写字符、数字、句点和短划线。
- 存储桶名称中的每个标签必须以小写字母或数字开头。
- 存储桶名称不能包含下划线、以短划线结束、包含连续句点或在句点旁边使用短划线。
- 存储桶名称不能采用 IP 地址格式 (198.51.100.24)。

API简要说明

➢ 列出所有的桶

```
/************************LIST BUCKET*********************************/
auto response = client.ListBuckets(); //直接调用API函数，得到返回对象
if (response.IsSuccess()) {
        auto buckets = response.GetResult().GetBuckets();
        for (auto iter = buckets.begin(); iter != buckets.end(); ++iter) {
                cout << iter->GetName() << "\t"  << endl;
        }
} else {
        cout << "Error while ListBuckets " << response.GetError().GetExceptionName()
                << " " << response.GetError().GetMessage() << endl;
}
/*********************************************************************/
```

➢ 编译并执行，结果如下

```
[root@master aws]#  ./a.out
aaaa
mybucket
wyz
```

由图可知，能列举出所有的桶

API简要说明

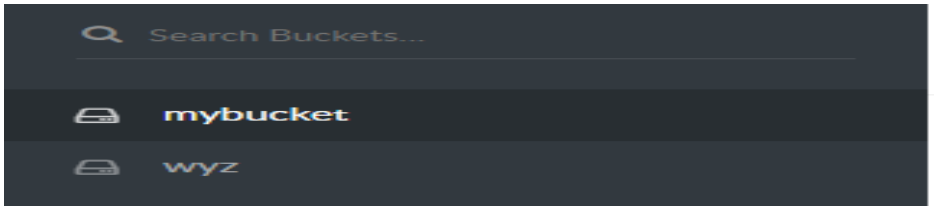
➢ 删除桶

```
/**************************DELETE BUCKET*****************************/
const Aws::String bucket_name = "aaaa"; //定义一个要删除的桶名
std::cout << "Deleting S3 bucket: " << bucket_name << std::endl;
Aws::S3::Model::DeleteBucketRequest bucket_request;//创建删除请求对象
bucket_request.SetBucket(bucket_name);//封装桶名
auto delcome = client.DeleteBucket(bucket_request);
if (delcome.IsSuccess())
{
        std::cout << "Done!" << std::endl;
}
else
{
        std::cout << "DeleteBucket error: "
                << delcome.GetError().GetExceptionName() << " - "
                << delcome.GetError().GetMessage() << std::endl;
}
/*********************************************************************/
```

➢ 编译并执行

```
[root@master aws]# ./a.out
Deleting S3 bucket: aaaa
Done!
[root@master aws]#
```

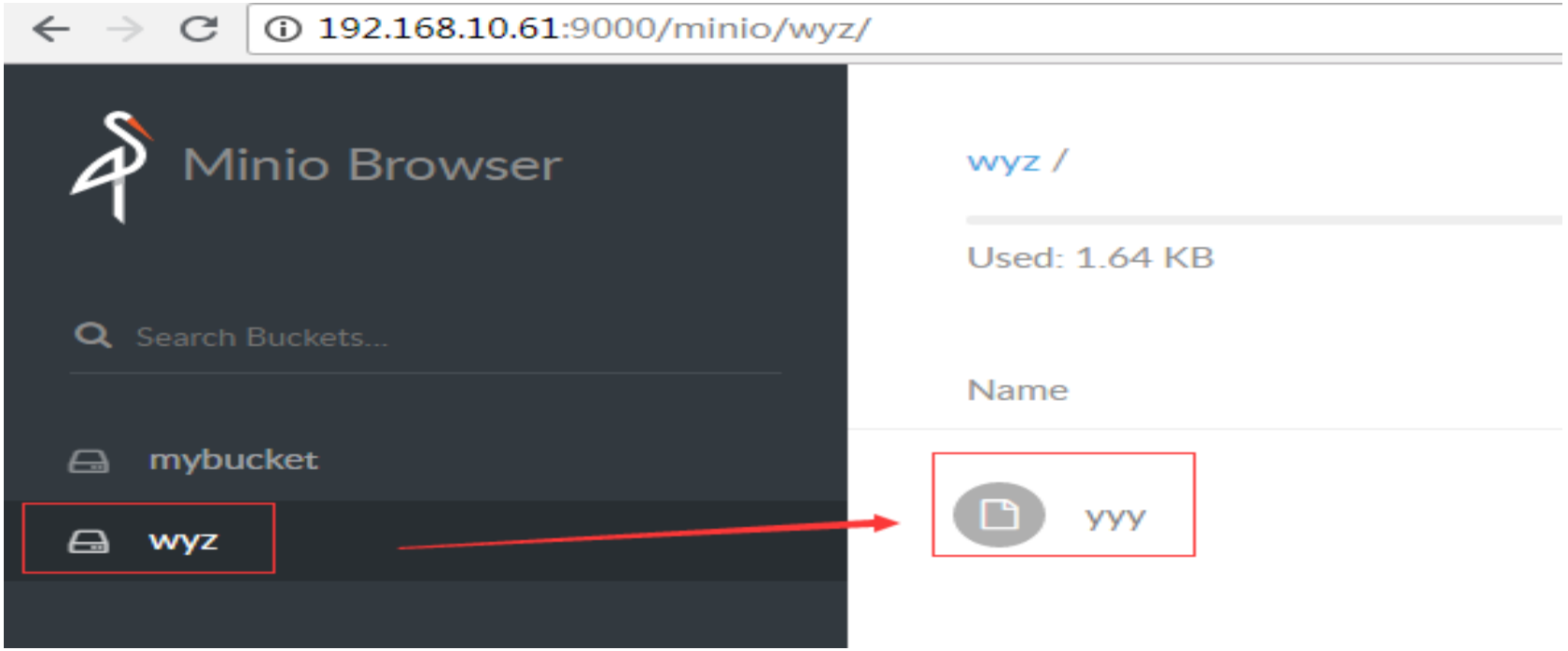
提示删除成功



刷新浏览器，也可以看到桶“aaaa”已经删除

API简要说明

➢ 删除桶的异常情况



桶“**wyz**”中有一个文件“**yyy**”,如果要删除这个桶的话，是无法删除的。提示信息如下：

```
[root@master aws]# ./a.out
Deleting S3 bucket: wyz
DeleteBucket error: BucketNotEmpty - Unable to parse ExceptionName: BucketNotEmpty Message: The bucket you tried to delete is not empty
[root@master aws]#
```

API简要说明

➢ 上传文件

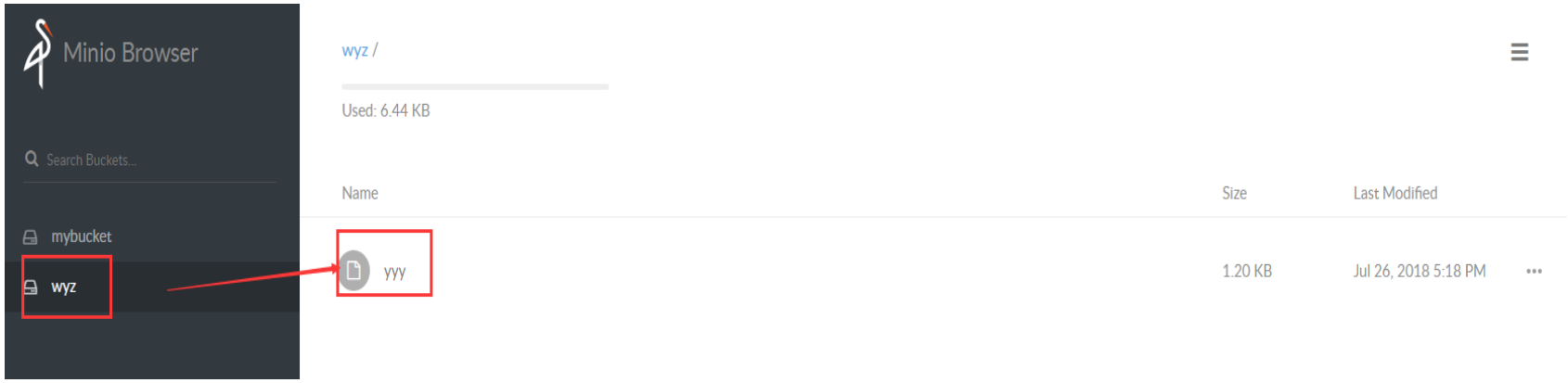
```
/**************************upload object*****************************/
const Aws::String bucket_name = "wyz";
const Aws::String key_name = "yyy";
const Aws::String file_name = "/home/wangyouzhan/minio-deployment.yaml";
std::cout << "Uploading " << file_name << " to S3 bucket " <<
        bucket_name << " at key " << key_name << std::endl;
Aws::S3::Model::PutObjectRequest object_request; //定义一个创建对象的请求
object_request.WithBucket(bucket_name).WithKey(key_name);//封装桶名和文件名
auto input_data = Aws::MakeShared<Aws::FStream>("PutObjectInputStream",
    file_name.c_str(), std::ios_base::in | std::ios_base::binary);//创建一个读取文件对象
object_request.SetBody(input_data);//封装请求
auto put_object_outcome = client.PutObject(object_request);//发送请求
if (put_object_outcome.IsSuccess())
{
        std::cout << "Done!" << std::endl;
}
else
{
        std::cout << "PutObject error: " <<
                put_object_outcome.GetError().GetExceptionName() << " " <<
                put_object_outcome.GetError().GetMessage() << std::endl;
}
/*********************************************************************/
```

API简要说明

- 编译并执行

```
[root@master aws]# ./a.out
Uploading /home/wangyouzhan/minio-deployment.yaml to S3 bucket wyz at key yyy
Done!
[root@master aws]#
```

- 界面查看



从上图可知，已经把/home/wangyouzhan/minio-deployment.yaml上传到minio服务器，且key值为“yyy”

API简要说明

➢ 关于对象键的命名规则

以下字符集通常可安全地用于键名称：

- 字母数字字符 [0-9a-zA-Z]
- 特殊字符 !、-、_、.、*、'、(和)

可能需要特殊处理的字符

(“&”)、 (“$”)、ASCII 字符范围 00–1F 十六进制 (0–31 十进制) 和 7F (127 十进制)、 (“@”)、 (“=”)、 (“;”)、 (“:”)、 (“+”)、空格 – 大量连续空格可能会在某些使用情形中丢失 (特别是多个空格)、 (“,”)、 (“?”)

要避免的字符

反斜杠 (“\”)、左大括号 (“{”)、不可打印的 ASCII 字符 (128–255 十进制字符)、插入符号 (“^”) 右大括号 (“}”)、百分比字符 (“%”)、重音符/反勾号 (“`”)、右方括号 (“]”)、引号、“大于”符号 (“>”)、左方括号 (“[”)、波浪字符 (“~”)、“小于”符号 (“<”)、字符 (“#”)、 (“|”)

API简要说明

➢ 上传文件异常情况

- 向一个不存在的桶中传文件

```
[root@master aws]# ./a.out
Uploading /home/wangyouzhan/minio-deployment.yaml to S3 bucket aaaa at key yyy
PutObject error: NoSuchBucket The specified bucket does not exist
[root@master aws]#
```

- 上传一个不存在的文件

```
[root@master aws]# ./a.out
Uploading /home/wangyouzhan/minio-deployment.yamlaa to S3 bucket wyz at key yyy
PutObject error:
```

经测试发现，上传一个不存在的文件时，提示出错，但没有给出具体的错误信息。

API简要说明

➢ 根据对象名下载文件

```
/**************************download object*******************************/
const Aws::String bucket_name = "wyz";
const Aws::String key_name = "yyy";
std::cout << "Downloading " << key_name << " from S3 bucket: " <<
    bucket_name << std::endl;
Aws::S3::Model::GetObjectRequest get_object_request; //创建请求对象
get_object_request.WithBucket(bucket_name).WithKey(key_name);//封装桶名和对象名
auto get_object_outcome = client.GetObject(get_object_request);//发送请求
if (get_object_outcome.IsSuccess())//如果成功则写文件
{
    Aws::OFStream local_file;
    local_file.open(key_name.c_str(), std::ios::out | std::ios::binary);
    local_file << get_object_outcome.GetResult().GetBody().rdbuf();
    std::cout << "Done!" << std::endl;
}
else
{
    std::cout << "GetObject error: " <<
        get_object_outcome.GetError().GetExceptionName() << " " <<
        get_object_outcome.GetError().GetMessage() << std::endl;
}
/*************************************************************************/
```

由上图代码可以看出来，是要从桶“wyz”里面下载key值为“yyy”的文件，注意红框中的参数，是要把下载下来的文件命名为“yyy”。

API简要说明

➢ 根据对象名下载文件

```
[root@master aws]# ./a.out
Downloading yyy from S3 bucket: wyz
Done!
[root@master aws]# ll yyy
-rw-r--r--. 1 root root 1233 Jul 26 17:34 yyy
[root@master aws]# md5sum /home/wangyouzhan/minio-deployment.yaml
07ab32ba477129672b138c1c92857996  /home/wangyouzhan/minio-deployment.yaml
[root@master aws]# md5sum yyy
07ab32ba477129672b138c1c92857996  yyy
[root@master aws]#
```

从上图可以看出，能够成功下载文件，文件和程序存放在同一目录下，通过md5sum命令对比源文件和新下载的文件，值相同。

API简要说明

➢ 根据对象名下载文件异常情况

- 要下载的对象不存在

```
[root@master aws]# ./a.out
Downloading yyy from S3 bucket: wyz
GetObject error: NoSuchKey The specified key does not exist.
[root@master aws]#
```

- 要下载的桶不存在

```
[root@master aws]# ./a.out
Downloading yyy from S3 bucket: aaaa
GetObject error: NoSuchBucket The specified bucket does not exist
[root@master aws]#
```

API简要说明

➢ 删除对象

```
/*********************************delete object************************/
const Aws::String bucket_name = "wyz";
const Aws::String key_name = "yyy";
std::cout << "Deleting" << key_name << " from S3 bucket: " <<
    bucket_name << std::endl;
Aws::S3::Model::DeleteObjectRequest object_request;//创建删除资源的请求对象
object_request.WithBucket(bucket_name).WithKey(key_name); //封装桶名和对象名
auto delete_object_outcome = client.DeleteObject(object_request);//发送请求
if (delete_object_outcome.IsSuccess())
{
    std::cout << "Done!" << std::endl;
}
else
{
    std::cout << "DeleteObject error: " <<
        delete_object_outcome.GetError().GetExceptionName() << " " <<
        delete_object_outcome.GetError().GetMessage() << std::endl;
}
/*************************************************************************/
```

从代码中可以看出，是要删除桶“wyz”中的“yyy”。

API简要说明

- 删除对象

```
[root@master aws]# ./a.out
Deletingyyy from S3 bucket: wyz
Done!
[root@master aws]#
```

- 浏览器观察结果



代码编译

- 编译命令

- g++ -g -std=c++11 -I/tmp/install/usr/local/include -L/tmp/install/usr/local/lib64 -laws-cpp-sdk-core -laws-cpp-sdk-s3 aws_minio.cpp

问题整理

➢ 是否支持异步上传和分块上传？

- 支持异步上传，分块上传

异步传输接口

```
void S3Client::PutObjectAsync(const PutObjectRequest& request, const PutObjectResponseReceivedHandler& handler, const std::shared_ptr<const Aws::C
{
  m_executor->Submit( [this, request, handler, context](){ this->PutObjectAsyncHelper( request, handler, context ); } );
}
```

- 异步分块上传

```
void S3Client::PutObjectAsync(const PutObjectRequest& request, const PutObjectResponseReceivedHandler& handler, const std::shared_ptr<const Aws::C
{
  m_executor->Submit( [this, request, handler, context](){ this->PutObjectAsyncHelper( request, handler, context ); } );
}
```

问题整理

- 是否标准HTTP？
  - 是标准HTTP
- 是否支持多IP？
  - 不支持，AWS内部使用域名做多IP解析
- aws-sdk-core 和 aws-sdk-s3依赖情况？
  - 不依赖外部库

谢谢！

中国科学院信息工程研究所
INSTITUTE OF INFORMATION ENGINEERING,CAS

谢谢！

Tel: 86-10-8254**** Fax: 86-10-8254****
E-mail: ***@iie.ac.cn
地址：北京市海淀区闵庄路甲89号 100093